Using objects in a scalar context gives similarly poor results:

```
var_dump("This is what happens when we stringify an object: " . new
stdClass());
```

```
Output:
Fatal error: Uncaught Error: Object of class stdClass could not be
converted to string
```

In all of these composite-to-scalar examples we have legal PHP syntax. Linting this code with `php -l` would not indicate any issues, though other tools would highlight the problem (a good IDE would draw attention to the fact that an array is an illegal operand type in addition).

It isn't difficult to see why composite values are problematic in scalar contexts; they are fundamentally different types of, well, types. Casting scalars to other scalar types, on the other hand, is quite common (that's how I started this section off, after all). This is where PHP can get a little wonky, and in fact some effort has been made in PHP 8 to reduce the edge cases that have in the past led to bugs and security vulnerabilities.

We've already seen that strings can behave like number types. A string is considered to be numeric if it meets certain criteria. Namely, if it contains only digit characters (and a single decimal point for floats), with the exception that the number can have any amount of leading or trailing spaces, tabs, or newline characters (line feed, carriage return, etc.). So `"123"` is numeric (`"123"` and `"123"` would be equivalent), and `"100 Dalmatians"` is not, albeit with a rather hefty caveat. Here's a more fleshed-out demonstration, which we'll run twice, once for PHP 7.4 and again for PHP 8:

```
var_dump(is_numeric("123"));
var_dump(is_numeric("100 Dalmatians"));
var_dump("123" + 1);
var_dump("123" + 0.1);
var_dump("100 Dalmatians" + 1);
var_dump("Five-hundred" + 1);
```

```
Output: (PHP 7.4)
bool(true)
bool(false)
int(124)
float(123.1)
```